Blind Low-Light sRGB Denoising by Fine-Tuning NAFNet

Team Voxel - Mora SP Cup 2026, Preliminary Round

1 Problem and scoring objective

The goal is to remove low-light noise from 992×992 8-bit sRGB photographs. The word *blind* means that the method is not told the noise level of each test image in advance. The official score is calculated for every image as

$$\text{composite} = 0.6\,\text{clip}\left(\frac{\Delta\text{PSNR}}{15}, 0, 1\right) + 0.4\,\max(\Delta\text{SSIM}, 0).$$

Before the PSNR term reaches its limit, a 1 dB improvement in ΔPSNR adds 0.04 to the composite score, while a 0.01 improvement in ΔSSIM adds 0.004. For this reason, we mainly optimized PSNR while still protecting image structure. The average brightness difference between each noisy image and its clean target is only 1.4 to 1.8 levels out of 255, which is very small compared with the noise level of about $\sigma \approx 28$. Therefore, our pipeline does not brighten the image, change gamma, or apply tone mapping.

2 Dataset observations

We first measured the noise in all 460 public noisy and clean image pairs. The full analysis can be reproduced from `analysis/noise_characterization/`.

- **Noise changes with image intensity.** The measured variance follows

  $$\text{var}(I) = aI + b,$$

  with $a = (4.36, 4.22, 4.86)$ and $b = (413.7, 369.6, 357.1)$ for the RGB channels. Since $\sqrt{b} \approx 19$ to 20, a strong sensor read-noise floor is already present in dark areas.
- **The noise level is different from image to image.** The estimated per-image σ ranges from 5.9 to 42.8 and does not form clear groups. The denoiser therefore has to work across a continuous range of noise strengths.
- **There are occasional extreme errors and repeated bad pixels.** Channel excess kurtosis is 1.86 to 2.36, which shows heavier tails than ordinary Gaussian noise. Random impulse-like errors affect about 0.033% of pixels per image, and 492 pixel locations appear as defects in 50 to 70% of frames.
- **Colour channels are not equally noisy.** Median robust noise is $Y = 17.98$, $Cb = 15.14$, and $Cr = 16.49$. Chroma is already quieter than luminance, so strong colour smoothing would remove useful detail.
- **Neighbouring noise samples are only weakly related.** Lag-1 residual autocorrelation is only 0.027 to 0.032. This means that complicated models for strongly correlated spatial noise are not necessary here.

3 Solution architecture

Main restoration network. We use NAFNet-width32 with encoder depths $[2, 2, 4, 8]$, 12 middle blocks, decoder depths $[2, 2, 2, 2]$, and 29,159,715 parameters. The encoder uses 32, 64, 128, 256, and 512 channels across four levels. NAFBlocks use SimpleGate and simplified channel attention instead of standard activation-heavy blocks. PixelShuffle is used for upsampling, skip connections bring fine image detail from the encoder to the decoder, and the network learns a residual that is added back to the input. Since

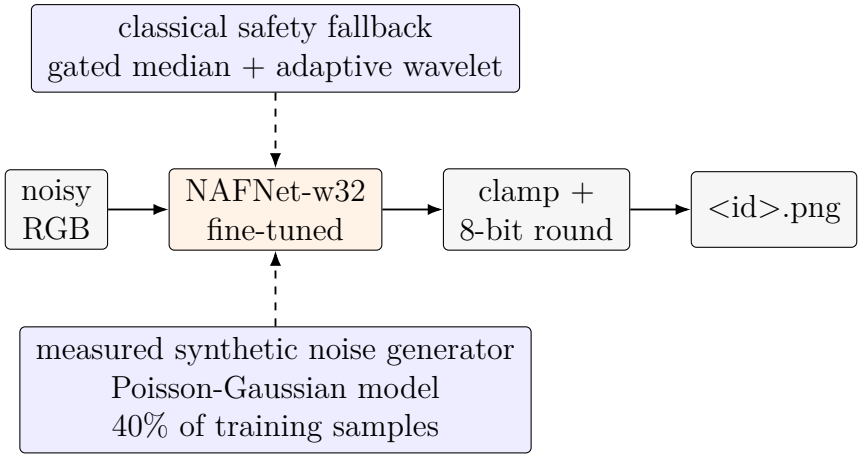


Figure 1: Submission pipeline. The solid path is the normal scored path. The measured noise model is used to create extra training examples, while the classical method is kept as a backup if the model cannot run.

$992 = 32 \times 31$, the whole image can be processed without padding, which avoids seam artifacts at the borders.

Synthetic noise based on the real dataset. Forty percent of training crops are created from clean images by adding noise that matches what we measured in the dataset. The generator uses the fitted $aI + b$ variance, a two-component Gaussian scale mixture to match the heavier tails, sparse impulse errors, and the 492 recurring sensor-defect positions. The severity scaling is constrained by $\mathbb{E}[s^2] = 1$, so it does not accidentally increase the overall variance. After generation, we measure the synthetic residual again and reject a generator setting if its variance or kurtosis is outside the measured range.

Safe inference. The preferred path processes the full 992×992 image and uses about 1.28 GB peak memory. If memory is insufficient, the runner automatically tries 512-pixel tiles, then 256-pixel tiles, both with smooth overlap blending. If the neural model still cannot run, it uses the classical backup. Input and output folders are kept separate, names are changed from `NNN_noise.png` to `NNN.png`, and every output is checked to be an RGB PNG of size 992×992.

4 Training methodology

We split the public data into 414 training images and 46 validation images. To make the validation set cover easy and difficult cases, the image IDs are sorted by estimated noise level and every tenth image is held out. The validation range is 7.57 to 40.66, compared with 5.86 to 42.83 for the full public set. The split is saved once and reused by training, validation, and ablation scripts. IDs 461 to 480 are never used for tuning.

Training uses 60% real noisy-clean crop pairs and 40% synthetic crops. Only D_4 geometric augmentation is used. The loss is per-image log-MSE, also called PSNRLoss, because the official metric averages PSNR per image. AdamW uses $\beta = (0.9, 0.9)$, weight decay 10^{-3}, a learning rate of 2×10^{-4}, 500 warm-up steps, and cosine decay to 10^{-6}. We also use FP16 mixed precision, gradient clipping at 1.0, and EMA decay 0.9995. The saved checkpoint is chosen using the official validation composite score after exact 8-bit rounding, not simply by the training loss.

5 Design decisions and alternatives

Every important choice was tested on the same validation split instead of being added only because it sounded useful.

Fine-tuning instead of training from scratch. Training throughput was about 0.9 Mpx/s on an RTX 4050. In 12 hours, this gives only about 4.6% of the compute used for

published NAFNet SIDD training. Starting from official SIDD weights is therefore much more practical. Even before fine-tuning, zero-shot NAFNet reaches a 0.3367 composite score, compared with 0.2633 for the organizer baseline.

No impulse prefilter before the network. A noise-gated adaptive median filter helps the standalone classical pipeline by 0.0152 composite, but it reduces the learned pipeline from 0.6433 to 0.6304. When the gate is made stricter, performance improves as fewer pixels are changed. This suggests that the trained network handles the injected impulse defects better while keeping small details. The median filter is therefore used only in the backup path.

No default test-time augmentation. Four-way and eight-way D_4 ensembles improve composite by only 0.0023 and 0.0031, while CPU runtime becomes 4.4 and 8.7 times longer. Since runtime contributes 15% of the grade, this small image-quality gain does not justify the extra cost in the submitted path.

Alternative	What the test showed	Effect
Adaptive median prepass	The network repairs impulse errors with less detail loss	−0.0129
Chroma smoothing	$Cb/Y = 0.84$, $Cr/Y = 0.92$; colour noise is already lower	−0.0012
Gamma 1.05	Noisy input and clean target already have matching exposure	−0.0432
Anscombe VST	Changes the image statistics seen by the pretrained network	−0.0778
TLC local pooling	Full-resolution checkpoint selection already handles this shift	−0.0014
Tiled 512 inference	Useful as a backup, but slightly changes global feature statistics	−0.0040
SCUNet or scratch training	Less suitable CPU profile or not enough training compute	Rejected
Hardcoded defect lookup	Would depend on fixed pixel IDs instead of general denoising	Forbidden

Table 1: Alternatives tested on the same validation split. The effect column is the change in composite score relative to the submitted configuration.

6 Validation protocol

All reported outputs are clipped to the valid pixel range, rounded to the exact 8-bit values that would be submitted, saved as PNG files, and scored with the unchanged `evaluation/evaluate.py`. We call the evaluator in two different ways, as a separate process and through its per-image functions, and both produce results that agree to 10^{-8}. Training metrics are also checked independently with scikit-image.

The clean validation targets are used only for evaluation. They are not used to estimate the noise model or to update the optimizer. Ground truth for IDs 461 to 480 is hidden, so we do not report PSNR, SSIM, or composite values for those preliminary test images.

The submitted configuration uses whole-image CPU inference. Tiled inference and TTA are reported as tested alternatives, not as hidden post-processing. Before submission, a clean-process preflight reruns inference and compares SHA-256 hashes with the final PNG files. We also rename inputs during testing to confirm that the algorithm follows image content rather than an image-ID-specific rule.

Configuration	PSNR	SSIM	ΔPSNR	Composite	s/image
Noisy input	19.82	0.4560	0.00	0.0000	–
Organizer baseline	24.34	0.6717	+4.51	0.2633	0.97
Pretrained NAFNet, zero-shot	24.80	0.7996	+4.98	0.3367	9.8
Classical fallback	26.12	0.7580	+6.30	0.3730	0.93
Fine-tuned NAFNet (submitted)	**31.63**	**0.9195**	**+11.81**	**0.6433**	**9.74**
Ours + TTA ×4	31.69	0.9203	+11.87	0.6456	42.5
Ours + TTA ×8	31.71	0.9207	+11.89	0.6464	84.5
Ours, tiled 512	31.47	0.9194	+11.65	0.6393	31.1

Table 2: Official metrics on the same 46 held-out images. Runtime is measured per 992×992 image on an Intel Core 5 210H using eight threads. The 20 preliminary images take 195 s in total.

7 Results

The submitted model improves over the organizer baseline by 7.29 dB PSNR, 0.2478 SSIM, and 0.3800 composite. Its composite score is 2.44 times the baseline. TTA gives a slightly higher image score, but the gain is very small compared with the large runtime increase, so it was not selected. Whole-image inference also beats the 512-tile backup by 0.0040 composite, which supports using the complete image whenever memory allows.

Ablation	PSNR	SSIM	Composite	Δ vs. submitted
Submitted, no prefilter or tone change	31.63	0.9195	0.6433	–
Gated adaptive median pre-pass	31.32	0.9162	0.6304	−0.0129
TLC local pooling	31.58	0.9193	0.6420	−0.0014
Chroma box smoothing	31.60	0.9194	0.6421	−0.0012
Gamma 1.05 brightening	30.30	0.9167	0.6001	−0.0432
Generalized Anscombe pre-net	29.51	0.9001	0.5655	−0.0778

Table 3: Ablation results using the identical validation set and official metric.

8 Interpretation

The results show that the best scored path is also the simplest one. Matching the synthetic training noise to the measured dataset helps the network handle different noise strengths. In contrast, several extra preprocessing steps remove useful detail or change brightness statistics and reduce the final score. The classical signal-processing work is still important because it tells us how to generate realistic training noise, gives a fast independent backup method, and provides measurable evidence for deciding which stages should stay and which should be removed.

9 Qualitative result and reproducibility

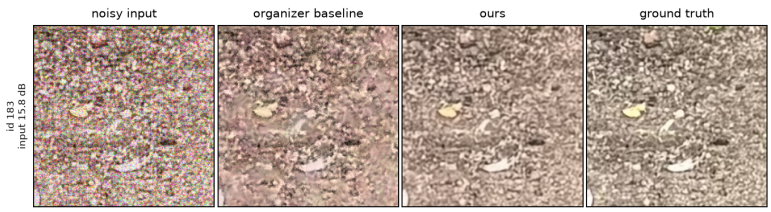


Figure 2: Centre crop of the hardest held-out image by estimated noise level. The model removes dense noise and repeated defects while keeping the original brightness, texture, and colour close to the clean target.

Once the checkpoint is installed, inference is deterministic and works fully offline. The default command is

```
python scripts/denoise.py -noise_dir <input> -denoised_dir <output>
```

The program loads the local checkpoint with a strict tensor-shape match, accepts any input filename ending in `_noise.png`, and writes only to the requested output folder. It also records SHA-256 values for the model, implementation, inputs, and outputs. A CPU backup path is always available, and optional acceleration does not change which model is selected. The README explains dependency installation, checkpoint location, the frozen model checksum, the exact command, and output verification. The submitted archive contains only `461.png` through `480.png`, all stored as RGB PNG files at 992×992.

Reproduction checklist.

1. Install `scripts/requirements.txt` using Python 3.11 or later.
2. Put the documented checkpoint in the local path specified by the README and verify its SHA-256 checksum before inference.
3. Run the command above from the repository root using the untouched noisy-image folder.
4. Run the preflight validator to confirm filenames, file count, format, dimensions, source integrity, deterministic hashes, and the absence of image-specific lookup rules.

The final held-out validation results are **31.63 dB PSNR**, **0.9195 SSIM**, and **0.6433 composite** across 46 images. These are validation results, not claims about the hidden preliminary test set. Every result table in this report uses the same split, the same 8-bit rounding, the same evaluator, and the same submitted configuration.

Backbone reference: S. Chen et al., "Simple Baselines for Image Restoration," ECCV 2022. Repository implementation, measurements, licenses, and verification tools are under `scripts/`.